# Contadores, microprocesador por dentro y assembly x86

Resumen de estudio de los videos #55 y #57 a #64 del curso **"Arquitectura y Lógica de Computadoras"** del canal Hacker Lykeion. Es la continuación de `logica-secuencial.md` y cubre el resto de los temas posteriores al primer parcial (clases 9 y 10 de la planilla).

| # | Video | Tema | Sección |
|---|---|---|---|
| 55 | Contador ascendente 4 bits | Contador con flip-flops T | 1 |
| 57 | Cómo funciona un microprocesador por dentro | Buses, ALU, registros, ciclo de instrucción | 2 |
| 58 | El manual de Intel | Arquitectura PC y compatibilidad | 3 |
| 59 | Modos de operación | Modo real, protegido, 64 bits | 4 |
| 60 | Registros del microprocesador Intel | Propósito general, segmento, flags, IP | 5 |
| 61 | Registros del microprocesador Intel 2 | Repaso sobre el manual, tamaños, flags | 5 |
| 62 | Instrucciones add sub imul idiv | De C a assembly, pila, aritmética | 6 y 7 |
| 63 | Llamado a funciones: call y ret | Funciones, paso de parámetros, retorno | 8 |
| 64 | Saltos condicionales: je, jne / jz, jnz | cmp, flags, saltos | 9 |

**Sobre este resumen**

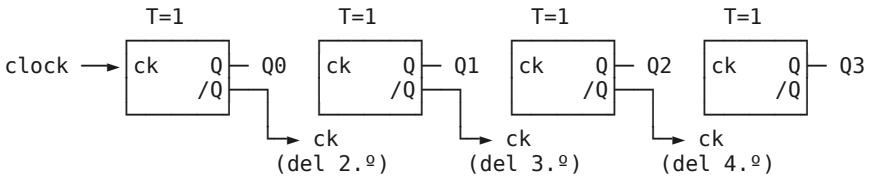
- Las notas marcadas con ⚠ señalan lapsus del video, o agregados que no están en el material original.
- Los listados de assembly están **reconstruidos a partir del relato** del profesor (en el video se ven en pantalla, pero la transcripción solo los describe). La lógica es la del video; algún detalle menor de escritura puede diferir.
- En la planilla figuran además "Cómo arranca la PC" y "Resolución de ejercicios" (clase 11) y los TP5. No tienen video enlazado, así que no están resumidos acá.

## 1. Contador ascendente de 4 bits

El video no explica cómo se diseña un circuito secuencial. Muestra **dónde se usan los flip-flops**.

### 1.1 Construcción

- **4 flip-flops T**.
- **Todas las entradas T están en 1**. Por lo tanto, cada flip-flop **invierte su salida** cada vez que recibe un disparo de clock.
- El **primer** flip-flop tiene su clock conectado a un **generador de pulsos** (el clock propiamente dicho).
- La salida **`Q̄` de cada flip-flop se conecta al clock del siguiente**.
- Las salidas son `Q0`, `Q1`, `Q2` y `Q3`.



### 1.2 Funcionamiento

- El primer flip-flop cambia en cada pulso del generador: `Q0` hace 0, 1, 0, 1… Su `Q̄` hace lo mismo pero invertido.
- Cuando un flip-flop pone su **`Q̄` en 1**, eso **dispara al siguiente**, que entonces invierte su salida.
- `Q̄` pasa a 1 justo cuando `Q` pasa de 1 a 0. Por eso cada etapa cambia cuando la anterior "da la vuelta" de 1 a 0, igual que el acarreo al contar.
- Resultado: las cuatro salidas forman un patrón que es **contar en binario de 0 a 15**.

### 1.3 Secuencia de conteo

Cada salida es un dígito de un número binario de 4 cifras. `Q0` es el bit menos significativo (el que cambia más rápido) y `Q3` el más significativo.

| Decimal | Q3 | Q2 | Q1 | Q0 | | Decimal | Q3 | Q2 | Q1 | Q0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | 8 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | | 9 | 1 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 0 | | 10 | 1 | 0 | 1 | 0 |
| 3 | 0 | 0 | 1 | 1 | | 11 | 1 | 0 | 1 | 1 |
| 4 | 0 | 1 | 0 | 0 | | 12 | 1 | 1 | 0 | 0 |
| 5 | 0 | 1 | 0 | 1 | | 13 | 1 | 1 | 0 | 1 |
| 6 | 0 | 1 | 1 | 0 | | 14 | 1 | 1 | 1 | 0 |
| 7 | 0 | 1 | 1 | 1 | | 15 | 1 | 1 | 1 | 1 |

- Con 4 bits se va de `0000` a `1111`: del 0 al 15, **16 números en total**.
- Después del 15 vuelve a 0 y sigue contando.

⚠ **Complemento (no está en el video):** este tipo de contador, donde solo el primer flip-flop recibe el clock y cada etapa dispara a la siguiente, se llama **contador asincrónico** o de rizado. Con `n` flip-flops cuenta de 0 a $2^n - 1$. Cada salida tiene la mitad de la frecuencia de la anterior (es el efecto del flip-flop T con T=1 visto en el resumen anterior).

## 2. Cómo funciona un microprocesador por dentro

### 2.1 Repaso: los buses (el micro visto desde afuera)

El microprocesador (CPU) se comunica con la memoria RAM y la memoria ROM a través de tres buses:

| Bus | Sentido | Para qué sirve |
|---|---|---|
| **De direcciones** | Un solo sentido: **sale** del CPU | Elegir la posición de memoria |
| **De datos** | Ida y vuelta | Al **escribir**, va del CPU a la RAM. Al **leer**, vuelve de la RAM o la ROM al CPU |
| **De control** | Entra y sale | Indicar si el dispositivo va a ser **leído o escrito** (la ROM solo se lee) y elegir a qué integrado se hace referencia |

Además de CPU, ROM y RAM aparece un bloque de **entrada/salida (I/O)**: es el mecanismo con el que el micro hace de interfaz con los dispositivos (discos rígidos y sus controladoras, almacenamiento en general, placas de red y de sonido, buses USB, serie, etc.).

## 2.2 Bloques internos del microprocesador

El video usa un diagrama en bloques **genérico**: algunos microprocesadores reales se parecen más a él y otros menos, pero sirve para explicar cualquiera.

**ALU (Unidad Aritmético-Lógica)**

- Es un bloque de hardware hecho con compuertas.
- Hace operaciones **aritméticas**: suma, resta, producto, división.
- Hace operaciones **lógicas** (comparaciones): igual, distinto, mayor, menor, mayor o igual, menor o igual.
- Todas las operaciones que uno escribe en un lenguaje de alto nivel sobre números enteros, y las comparaciones, las resuelve la ALU.

**Registros**

- Un registro es un **conjunto de flip-flops agrupados** que guarda bits. Ejemplo: 8 flip-flops forman un registro de 8 bits; cada uno guarda un 0 o un 1.
- Analogía: es **como una variable, pero por hardware**. Está **dentro del microprocesador**, no en la memoria RAM.
- Un micro tiene muchos registros:
  - **De dirección**: guardan direcciones de memoria.
  - **Program counter** (contador de programa): así se llama en algunas arquitecturas al registro que indica por dónde va el programa.
  - **De propósito general**: en la arquitectura PC se llaman A, B, C y D (hay algunos más).
  - **Acumulador**.
  - **Registro de instrucción**: el lugar donde se guarda la instrucción que se va a ejecutar.
  - **IP (instruction pointer)**: en los micros de PC, apunta a la dirección de memoria de la instrucción a ejecutar.

**Lógica de control (unidad de decodificación)**

- **Decodifica** la instrucción que llegó desde la memoria.
- Según el código de cada instrucción, resuelve cómo ir a buscar los datos y qué bloques usar.

⚠ En el video se dice que "el registro de instrucción en los micros PC lo llamamos IP". Conviene separar las dos ideas: el **registro de instrucción** guarda la instrucción que se trajo de memoria; el **IP** guarda la dirección de la instrucción a ejecutar.

## 2.3 Ejemplo: una instrucción de assembly

```
ADD AX, BX        ; AX = AX + BX
```

- Suma el registro AX con el registro BX y **guarda el resultado en AX**.
- Esa línea de assembly **representa un código numérico** (código de operación), por ejemplo un valor hexadecimal. En el video se inventa uno (`0xF5`) solo para ilustrar.
- Ese código es lo que realmente está guardado en la memoria RAM y lo que decodifica la lógica de control.

## 2.4 Cómo se ejecuta una instrucción (paso a paso)

1. **Traer la instrucción.** El IP apunta a una dirección de memoria RAM. La instrucción guardada ahí viaja por el **bus de datos** hacia el microprocesador.
2. **Decodificar.** La lógica de control interpreta el código de la instrucción.
3. **Buscar los operandos.** Según la instrucción:
   - puede haber que ir a la RAM a buscarlos (usando los buses de datos, direcciones y control), o
   - pueden estar ya en registros del micro (cargados en un paso anterior o resultado de una operación previa).
   - Hay distintos **tipos de direccionamiento**.
4. **Operar.** Los datos se procesan en la **ALU** y se calcula un resultado.

5. **Guardar el resultado.** Suele quedar en un registro de propósito general (por ejemplo EAX). Luego se puede escribir en alguna dirección de RAM a través de los buses, o dejarlo en el registro.
6. **Repetir.** Debajo de esa línea hay otra, y el micro hace los mismos pasos, una instrucción tras otra.

# 3. El manual de Intel y la arquitectura PC

- El manual de Intel es "la Biblia" del microprocesador: un PDF de **casi 5000 páginas**, dividido en volúmenes.
- La **arquitectura PC está basada en un microprocesador Intel**. Cuando IBM hizo la computadora personal eligió un chip de Intel.
- La razón: Intel **prometió que la arquitectura se mantendría** en los micros siguientes. Así, los micros nuevos podrían **ejecutar código ya escrito** para los anteriores (compatibilidad hacia atrás).
- Dato del profesor para dimensionar el manual: leyó como mucho un 30 %, entendió un 25 % y usó un 5 %.
- De todo el manual, lo principal para empezar es: **los modos de operación** y **los registros**. Ambos están en el Volumen 1, capítulo 3.

# 4. Modos de operación del microprocesador

## 4.1 Contexto

- Los micros actuales son de **64 bits** (desde hace más de 10 años, contando desde 2019, cuando se grabó el video).
- Pero los micros de **arquitectura PC** pueden funcionar en **16, 32 y 64 bits**.
- Vale para Intel y para AMD: un micro AMD funciona igual.

## 4.2 Los modos

| Modo | Bits | Cuándo se usa | Características |
|---|---|---|---|
| **Modo real** | 16 | Al **encender** la computadora o apretar **reset** | El micro funciona como un **8086**. **No** se puede acceder a toda la memoria RAM |
| **Modo protegido** | 32 | Cuando corre un sistema operativo (Windows, Linux) | El acceso a la memoria está **protegido** por dos mecanismos: **segmentación** y **paginación**. Se puede acceder a toda la RAM |
| **System management mode** | — | Administración del microprocesador | No es el modo de ejecución normal |
| **Modo compatibilidad** | 64 (corre 32) | Dentro de la arquitectura de 64 bits | Estando en 64 bits, permite correr **código de 32 bits** |
| **Modo 64 bits** | 64 | Dentro de la arquitectura de 64 bits | El micro funciona como un micro de 64 bits |

- Cita del manual: "the processor is placed in real-address mode following power-up or a reset". Es decir, **el micro siempre arranca en modo real**.
- El manual agrupa los modos en dos partes: los modos básicos (real, protegido y system management) y, aparte, la arquitectura Intel 64 (compatibilidad y 64 bits).

### 4.3 Quién inventó los 64 bits de la PC

- La arquitectura de 64 bits que usamos **la inventó AMD**, no Intel.
- AMD hizo una arquitectura de 64 bits **compatible con el código viejo**.
- Intel, en los años 2000, **se apartó de su arquitectura tradicional** e hizo una arquitectura nueva de 64 bits, intentando romper el paradigma.
- **El mercado no la aceptó** y adoptó el modelo de AMD, que seguía la arquitectura original. Intel tuvo que tomarlo.
- Por eso existen los dos modos de 64 bits: compatibilidad con 32 y 64 puro.

⚠ **Complemento (no está en el video):** la arquitectura nueva de Intel que no prosperó se llamó Itanium (IA-64). La de AMD se conoce como AMD64 o x86-64.

## 5. Registros del microprocesador (arquitectura PC)

Para programar en assembly o conocer la arquitectura de cualquier micro, **hay que aprender los nombres de sus registros y para qué sirve cada uno**. Sin eso no se puede escribir nada.

En el manual de Intel: Volumen 1, capítulo 3, sección 3.4 (Basic Program Execution Registers). Hay cuatro grupos:

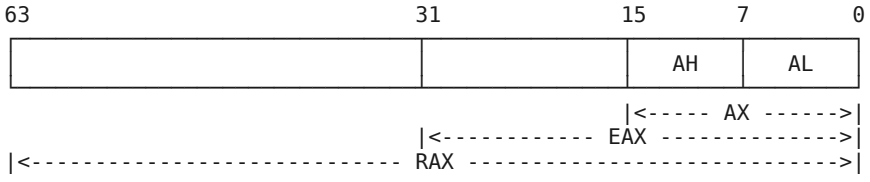
1. Registros de **propósito general**.
2. Registros de **segmento**.
3. Registro de **flags** (estado).
4. **Instruction pointer**.

Intel y AMD usan los mismos nombres, porque es la misma arquitectura hecha por dos fabricantes. (Existen además otros registros, propios de cada fabricante, asociados a otros tipos de instrucciones.)

### 5.1 Cómo se nombran según el tamaño

Los nombres vienen "desde la antigüedad" y se fueron extendiendo:

| Tamaño | Nombre (registro A) | Regla | Tipo de dato |
|---|---|---|---|
| 8 bits, parte baja | **AL** | L = low | byte |
| 8 bits, parte alta | **AH** | H = high | byte |
| 16 bits | **AX** | AH + AL (8 + 8) | word |
| 32 bits | **EAX** | E = extended (se agregan 16 bits más) | double word |
| 64 bits | **RAX** | R adelante | quad word |




- Todo lo anterior vale igual para **A, B, C y D**.
- Según cuántos bits se quieran tomar del registro, se escribe AL, AH, AX, EAX o RAX.
- La "granularidad" es: los primeros 8 bits (AL), los segundos 8 bits (AH) o los primeros 16 (AX). Al pasar a 32 bits se pasó directo a EAX.
- **SI, DI, BP y SP** nacieron de 16 bits (no se parten en H y L). También tienen sus versiones E (32 bits) y R (64 bits).
- En 64 bits aparecen **algunos registros más** de propósito general.

**Tamaños de dato** (aparecen en el assembly):

| Nombre | Bits |
|---|---|
| byte | 8 |
| word | 16 |
| double word (dword) | 32 |
| quad word (qword) | 64 |

## 5.2 Registros de propósito general

| Registro (32 bits) | Nombre | Uso típico |
|---|---|---|
| **EAX** | Acumulador | Operaciones aritméticas; ahí suelen quedar los resultados |
| **EBX** | Base | Base en el direccionamiento de memoria (puntero a datos) |
| **ECX** | Contador | Operaciones de repetición (como un `while` en assembly): guarda la cantidad de veces que se hace algo |
| **EDX** | Datos | Datos; entrada/salida |
| **ESI** | Source index (índice fuente) | Puntero de **origen** al copiar bloques de memoria o cadenas |
| **EDI** | Destination index (índice destino) | Puntero de **destino** en esas copias |
| **EBP** | Base pointer (puntero base) | Base de la pila; referencia para ubicar datos |
| **ESP** | Stack pointer (puntero de pila) | Apunta a la pila |

Para qué se pueden usar, según el manual:

- Como **operandos** de operaciones lógicas y aritméticas.
- Como operandos para **calcular direcciones** (por ejemplo: "la dirección que está en tal registro, más un número").
- Como **punteros a memoria**.

Detalles:

- **ESI y EDI**: hay instrucciones del micro que copian un string o un bloque de memoria usando la dirección guardada en ESI como origen y la guardada en EDI como destino.
- **EBP y ESP** se pueden usar como **base más desplazamiento**: "a partir de esta dirección, correte tantos bytes en la memoria".
- Hay muchísimos modos de direccionamiento; esto es una simplificación.

**El puntero de pila (SP / ESP / RSP) es importantísimo**

- Se llama SP en 16 bits, ESP en 32 y RSP en 64.
- **Muchísimos ataques de software** que buscan tomar el control de un sistema operativo apuntan a **modificar el valor de este puntero**.
- Motivo: al llamar a una función, subrutina o interrupción, el micro usa la pila para saber adónde volver. Si alguien logra cambiar ese valor, al retornar el micro no vuelve a lo que estaba haciendo sino **al lugar que el atacante eligió**. En ese momento tomó el control.
- Hay que aprender bien cómo funciona en cada modo: hay diferencias entre 16, 32 y 64 bits, y según el **nivel de privilegio** (código de usuario o del sistema) la pila está en un lugar distinto.

⚠ En el video se dice "en 8 bits SP". Es un lapsus: SP es de **16 bits**.

⚠ **Complemento (no está en el video):** los registros adicionales de 64 bits son R8 a R15.

## 5.3 Registros de segmento

- Vienen de cuando el micro funcionaba en 16 bits.
- Guardan la referencia a un **segmento** de memoria, para hacer un direccionamiento de **segmento + desplazamiento**.
- **Siempre son de 16 bits**, aunque el micro funcione en 32 o en 64.

| Registro | Nombre | A qué segmento hace referencia |
|---|---|---|
| **CS** | Code segment | Donde está el **código** |
| **DS** | Data segment | Donde están los **datos** con los que opera el programa |
| **SS** | Stack segment | La **pila** (tiene su propio segmento) |
| **ES, FS, GS** | Adicionales | Segmentos de **datos auxiliares** |

Qué contienen según el modo:

- **Modo real (16 bits)**: los 16 bits sirven directamente para el direccionamiento de base más desplazamiento.
- **Modo protegido y modos de 64 bits**: el registro contiene un **índice dentro de un vector (tabla) de descriptores de segmento**. Cada descriptor describe un bloque de memoria RAM (de código, de datos o de pila) con su **inicio, fin, tamaño y permisos** (lectura, sentido de crecimiento, etc.).
- En 64 bits la lógica es parecida a la del modo protegido, con algunos cambios.

## 5.4 Registro de flags (registro de estado)

- En otros micros y microcontroladores se llama **registro de estado**, porque representa el estado de funcionamiento del microprocesador en un momento dado.
- En Intel se llama **flags** (banderas): cada bit es una bandera que "se levanta" para avisar que pasó algo.
- Nombres: **FLAGS** (16 bits), **EFLAGS** (32 bits), **RFLAGS** (64 bits).

Ejemplos de flags:

| Flag | Qué indica |
|---|---|
| **Zero flag** | La última operación dio **cero** |
| **Sign flag** | El bit más significativo del resultado quedó en 1 (el resultado es **negativo**) |
| **Carry flag** | Hubo **acarreo** (carry) o préstamo (borrow) en una operación |
| **Overflow flag** | Hubo **desborde** |
| **Parity flag** | Paridad de los primeros 8 bits del resultado |
| **Interrupt flag** | Si las **interrupciones** están habilitadas o deshabilitadas |

- Hay bits que tienen **siempre el mismo valor** (fijos en 0 o en 1); eso se respeta en toda la arquitectura.
- Para qué sirven al programar: después de ejecutar una instrucción se puede **consultar el valor de esos bits y tomar una decisión**. Las instrucciones de **salto condicional** hacen exactamente eso (sección 9).

## 5.5 Instruction pointer

- Es **el más importante de todos**.
- Guarda **la dirección de memoria de la próxima instrucción a ejecutar**.
- Nombres: **IP** (16 bits), **EIP** (32 bits, extended), **RIP** (64 bits).

### 5.6 Cuadro resumen de registros

| Grupo | 16 bits | 32 bits | 64 bits | Función |
|---|---|---|---|---|
| Propósito general | AX, BX, CX, DX | EAX, EBX, ECX, EDX | RAX, RBX, RCX, RDX | Operandos, direcciones, punteros |
| Índices | SI, DI | ESI, EDI | RSI, RDI | Origen y destino en copias |
| Pila | BP, SP | EBP, ESP | RBP, RSP | Base y tope de la pila |
| Segmento | CS, DS, SS, ES, FS, GS | (siguen siendo de 16) | (siguen siendo de 16) | Código, datos, pila, auxiliares |
| Estado | FLAGS | EFLAGS | RFLAGS | Estado de la última operación y del micro |
| Instrucción | IP | EIP | RIP | Dirección de la próxima instrucción |

## 6. De C a assembly: conceptos previos

### 6.1 La cadena de traducción

1. **Código C** (alto nivel).
2. **Código assembly** (lo genera el compilador).
3. **Código de máquina**: código binario.
   - Se lo llama **binario** cuando se carga en un microprocesador o microcontrolador.
   - Se lo llama **ejecutable** cuando es ese binario más lo que necesita un sistema operativo para poder ejecutarlo.

### 6.2 Cómo se ve un archivo de assembly

- **Directivas**: líneas del principio que son indicaciones para el compilador/ensamblador. No son instrucciones del micro.
- **Etiquetas**: un nombre seguido de dos puntos (por ejemplo `main:`). **Cada función de C es una etiqueta en assembly.** Una etiqueta corresponde a una dirección de memoria.
- **Instrucciones**: el código propiamente dicho.

### 6.3 Cómo generar el assembly con gcc

En el video se compila en Linux con **gcc**, con estas opciones:

```
gcc -S -O0 -masm=intel -fverbose-asm programa.c
```

| Opción | Para qué sirve |
|---|---|
| `-S` | Genera el código assembly en vez del binario |
| `-O0` | **No optimiza**. Los compiladores optimizan para usar poca RAM o correr rápido; sin optimización el assembly sale lo más básico posible y es fácil de seguir |
| `-masm=intel` | Usa la **sintaxis de Intel**. Por defecto gcc usa la de **AT&T**, que escribe las asignaciones "al revés". La de Intel es la más común |
| `-fverbose-asm` | Intercala, como comentarios, las líneas de C junto al assembly que les corresponde |
| `programa.c` | El archivo fuente |

- El assembly que sale corresponde al micro y sistema donde se compila: en el video, **64 bits**.
- En la **sintaxis Intel**, el orden es `instrucción destino, origen`. Ejemplo: `mov eax, 5` guarda un 5 en EAX.

⚠ La transcripción no deletrea las opciones; la línea de arriba es la forma estándar de lo que el profesor describe.

### 6.4 Memoria de un programa: pila y heap

Es un tema que cuesta y "en el que se equivocan en los exámenes". Las variables de un programa pueden estar en dos lugares:

| | **Pila (stack)** | **Heap ("parva" o montículo)** |
|---|---|---|
| Qué variables | Las **creadas en el código**: se les reserva memoria al escribir el programa. Ejemplo: `int a = 4;` | Las que se crean **dinámicamente**, durante la ejecución |
| Cómo se piden | Automáticamente | En C: con `malloc`. En Java: con `new` y el constructor de una clase |
| Cuánto viven | Al **salir de la función** (o terminar el proceso), la variable **se pierde** | Hasta que se libera |
| Quién libera | Nadie; es automático | En C: **el programador** debe pedirla y liberarla. En Java: el **garbage collector** libera la que ya no se usa |

- La memoria de un proceso tiene la pila, el heap y otros sectores.
- **El sistema operativo le asigna a cada programa una dirección de pila** cada vez que corre. En el video se usa `0x7D00` como ejemplo inventado.
- Los ejemplos del curso están hechos **totalmente en pila**: todas las variables viven en la pila.

### 6.5 Idea clave: los nombres de las variables no existen

- En assembly no hay variables con nombre. Los nombres son **"nombres de fantasía"** del lenguaje de alto nivel.
- Cada variable es en realidad **una dirección de memoria**, expresada como **el puntero base de pila (RBP) más un desplazamiento**: `[rbp-4]`, `[rbp-8]`, `[rbp-12]`.
- Esas direcciones **cambian en cada ejecución** del programa.
- Un `int` de C ocupa **32 bits** = 4 posiciones de memoria (4 bytes). En assembly aparece como `DWORD PTR`.

## 7. Instrucciones aritméticas: add, sub, imul, idiv

### 7.1 El programa en C

Tres variables enteras declaradas e inicializadas, y cuatro operaciones:

```
int main() {
    int val1 = 5;
    int val2 = 10;
    int resu = 0;

    resu = val1 + val2;   /* suma           */
    resu = val1 - val2;   /* resta          */
    resu = val1 * val2;   /* multiplicación */
    resu = val2 / val1;   /* división       */
}
```

## 7.2 Prólogo de la función y carga de variables

```
main:
    push rbp                    ; guarda en la pila el RBP de quien nos llamó
    mov  rbp, rsp               ; RBP pasa a ser la base de la pila de esta función
    mov  DWORD PTR [rbp-12], 5  ; val1 = 5
    mov  DWORD PTR [rbp-8], 10  ; val2 = 10
    mov  DWORD PTR [rbp-4], 0   ; resu = 0
```

- **`push rbp`**: guarda el valor del puntero base de pila en la pila. No sabemos adónde apuntaba; lo importante es que quedó guardado para poder recuperarlo al final.
- **`mov rbp, rsp`**: cuando se ejecuta una función, la pila deja de referirse a la función que llamó y pasa a referirse a la que se ejecuta ahora. Antes, el puntero tenía el valor de la pila del sistema operativo (que lanzó el programa).
- A partir de ahí, **todo se ubica respecto de RBP**.
- `mov DWORD PTR [rbp-12], 5`: mueve a la memoria, 12 lugares "hacia abajo" de RBP, el valor 5 representado en 32 bits (`101` completado con ceros a la izquierda).

**Mapa de la pila** (tomando `0x7D00` como base):

| Desplazamiento | Dirección | Contenido | Variable |
|---|---|---|---|
| 0 | `0x7D00` | RBP guardado | (base de la pila) |
| −4 | `0x7CFC` | 0 | `resu` |
| −8 | `0x7CF8` | 10 | `val2` |
| −12 | `0x7CF4` | 5 | `val1` |

Cada variable ocupa 4 "cajoncitos" (4 bytes).

⚠ Al explicar `mov rbp, rsp`, el relato del video lo cuenta al revés ("apuntar el stack pointer al valor de RBP"). En sintaxis Intel el destino va primero: se copia **RSP en RBP**. En el video de `call` y `ret` lo dice correctamente: "se carga el nuevo puntero base de pila, que está guardado en el puntero a pila".

## 7.3 Suma: `add`

```
    mov  edx, DWORD PTR [rbp-12]   ; EDX = val1
    mov  eax, DWORD PTR [rbp-8]    ; EAX = val2
    add  eax, edx                  ; EAX = EAX + EDX
    mov  DWORD PTR [rbp-4], eax    ; resu = EAX
```

Paso a paso:

1. **El microprocesador no opera con lo que está en la RAM: hay que traerlo a registros.** Se trae `val1` a EDX y `val2` a EAX.
2. `add` suma los dos registros. **El resultado queda en EAX** (el registro acumulador).
3. El resultado está dentro del micro, pero lo queremos en `resu`, que está en la RAM. Se **mueve de EAX a `[rbp-4]`**.

## 7.4 Resta: `sub`

```
    mov  eax, DWORD PTR [rbp-12]   ; EAX = val1
    sub  eax, DWORD PTR [rbp-8]    ; EAX = EAX - val2
    mov  DWORD PTR [rbp-4], eax    ; resu = EAX
```

- Diferencia con la suma: acá la operación se hace **entre un registro y un valor que está en memoria**, sin traerlo antes a otro registro.
- Algunas instrucciones admiten un operando en memoria y otras no. En este caso, sí.

## 7.5 Multiplicación: `imul`

```
    mov  eax, DWORD PTR [rbp-12]   ; EAX = val1
    imul eax, DWORD PTR [rbp-8]    ; EAX = EAX * val2   (5 * 10)
    mov  DWORD PTR [rbp-4], eax    ; resu = EAX
```

- El producto queda en EAX y luego se pasa a `resu`.

## 7.6 División entera: `idiv`

```
    mov  eax, DWORD PTR [rbp-8]    ; EAX = val2  (dividendo)
    idiv DWORD PTR [rbp-12]        ; divide por val1 (divisor)
    mov  DWORD PTR [rbp-4], eax    ; resu = EAX  (cociente)
```

- El uso "suena raro" comparado con las otras: **el dividendo se carga antes en EAX**, y a `idiv` solo se le pasa **el divisor** (acá, una dirección de memoria).
- Después de `idiv` la división ya está hecha: **el cociente queda en EAX**.

⚠ **Complemento (no está en el video):** `idiv` divide el número formado por EDX:EAX. Por eso gcc pone antes una instrucción `cdq`, que extiende el signo de EAX hacia EDX. El cociente queda en EAX y el **resto** en EDX.

## 7.7 Final de la función

```
    mov  eax, 0                    ; valor de retorno: 0
    pop  rbp                       ; recupera el RBP guardado al principio
    ret                            ; vuelve a quien llamó a main
```

- **`mov eax, 0`**: corresponde a la **llave de cierre** de la función. El valor que devuelve una función **se devuelve en EAX**; acá se devuelve 0.
- **`pop rbp`**: es la operación análoga a `push rbp`. `push` puso el valor en la pila; `pop` lo recupera.
- **`ret`**: retorna a la función que llamó a esta (en el caso de `main`, al sistema operativo).

## 7.8 Cuadro de instrucciones vistas

| Instrucción | Qué hace | Ejemplo |
|---|---|---|
| `mov destino, origen` | Copia un valor (registro, memoria o literal) | `mov eax, DWORD PTR [rbp-8]` |
| `add a, b` | `a = a + b` | `add eax, edx` |
| `sub a, b` | `a = a - b` | `sub eax, DWORD PTR [rbp-8]` |
| `imul a, b` | `a = a * b` (enteros con signo) | `imul eax, DWORD PTR [rbp-8]` |
| `idiv b` | Divide EAX por `b`; cociente en EAX | `idiv DWORD PTR [rbp-12]` |
| `push r` | Guarda un valor en la pila | `push rbp` |
| `pop r` | Recupera un valor de la pila | `pop rbp` |
| `ret` | Retorna de la función | `ret` |

**Patrón general de una operación:** traer los operandos de la memoria a registros → operar (el resultado queda en EAX) → guardar EAX en la variable de destino.

# 8. Llamado a funciones: `call` y `ret`

## 8.1 El programa en C

```
int mi_funcion_duplicadora(int b) {
    return b * 2;
}

int main() {
    int x = 4;
    int r = 0;
    r = mi_funcion_duplicadora(x);
}
```

- `main` tiene dos variables en la pila: `x` (vale 4) y `r` (vale 0).
- A la función se le pasa el valor de `x`. **El 4 se pasa por copia.**
- La función lo recibe en su variable local `b`, que vive en la pila **de esa función**. Lo multiplica por 2 y retorna el resultado.

## 8.2 Assembly de `main`

```
main:
    push rbp                        ; guarda el puntero base de pila
    mov  rbp, rsp                   ; carga el nuevo puntero base
    sub  rsp, 16                    ; desplaza el puntero de pila 16 lugares
    mov  DWORD PTR [rbp-8], 4       ; x = 4
    mov  DWORD PTR [rbp-4], 0       ; r = 0
    mov  eax, DWORD PTR [rbp-8]     ; EAX = x
    mov  edi, eax                   ; EDI = EAX  (parámetro para la función)
    call mi_funcion_duplicadora     ; salta a la función, con retorno
    mov  DWORD PTR [rbp-4], eax     ; r = EAX  (valor devuelto)
    mov  eax, 0                     ; valor de retorno de main
    leave                           ; restaura la pila
    ret
```

- **`sub rsp, 16`**: mueve el puntero de pila para reservar el lugar de las variables locales, de modo que no se pisen con lo que apile la función llamada.
- El 4 se copia de la pila a EAX y de EAX a **EDI**: el parámetro viaja **en un registro**.

## 8.3 Assembly de la función

```
mi_funcion_duplicadora:
    push rbp                        ; guarda el RBP de main
    mov  rbp, rsp                   ; nueva base de pila, para esta función
    mov  DWORD PTR [rbp-4], edi     ; b = EDI  (copia el parámetro a su propia pila)
    mov  eax, DWORD PTR [rbp-4]     ; EAX = b
    add  eax, eax                   ; EAX = EAX + EAX  (b * 2)
    pop  rbp                        ; recupera el RBP de main
    ret                             ; vuelve a la línea siguiente al call
```

- Empieza igual que siempre: guarda RBP y fija su propia base de pila.
- **El compilador, en vez de multiplicar por 2, suma EAX consigo mismo.** Da el mismo resultado.
- El resultado queda en **EAX**.

## 8.4 Recorrido completo del valor

1. `x` (un 4) está en la pila de `main`, en `[rbp-8]`.
2. Se copia a **EAX** y de ahí a **EDI**.
3. **`call`**: el micro **salta, con retorno**, a la etiqueta `mi_funcion_duplicadora`.
4. La función copia **EDI** a su propia pila (`[rbp-4]`): esa es la variable `b`.
5. Carga `b` en EAX y lo duplica. El 8 queda en **EAX**.
6. **`ret`**: el micro vuelve a la línea **siguiente** al `call`.
7. De vuelta en `main`, se copia **EAX** a `[rbp-4]`: esa es la variable `r`.

### 8.5 Ideas clave

- `call`: instrucción para llamar a subrutinas o funciones. Es un **salto con retorno** a una etiqueta. Para seguir el programa hay que buscar esa etiqueta en el código y ver qué hace.
- `ret`: retorna al lugar desde donde se llamó a la función.
- **Paso por copia**: la función trabaja sobre una copia del valor; la variable original no se toca.
- **El valor de retorno queda en EAX.** Que el retorno vaya en EAX (o en otros registros) y que el parámetro vaya en EDI es una **convención** del compilador y del sistema operativo para el que se compila.
- Si uno escribe sus propias funciones en assembly, se hacen igual: una **etiqueta con dos puntos**, `pop` para recuperar el valor de la pila y `ret` para retornar.

⚠ **Complemento (no está en el video):** `call` guarda en la pila la dirección de la instrucción siguiente (la dirección de retorno), y `ret` la saca de la pila para saltar ahí. Por eso modificar la pila permite desviar el retorno: es el ataque mencionado en la sección 5.2. La convención usada en Linux de 64 bits se llama System V AMD64 ABI.

## 9. Saltos condicionales: `cmp`, `je`, `jne`

Un **salto condicional** es un salto que ocurre **solo si se cumple una condición**. Es la forma en que assembly implementa las estructuras de control (`if`, bucles).

### 9.1 Mecanismo: comparar y después saltar

Son siempre dos pasos:

1. `cmp a, b` compara dos valores (registro, memoria o literal).
   - **No devuelve ningún valor.**
   - Lo que hace es modificar bits del **registro de flags** (EFLAGS en 32 bits, RFLAGS en 64).
   - Si los valores son **iguales**, un bit de ese registro se pone en **1**. Si son distintos, queda en **0**.
2. Una **instrucción de salto condicional** (las que empiezan con **J** seguidas de letras) mira ese bit y decide:
   - si la condición se cumple, **salta a la etiqueta** indicada;
   - si no se cumple, **sigue con la línea de abajo**.

| Instrucción | Significado | Salta si… |
|---|---|---|
| `je etiqueta` | jump if equal | la comparación dio **igual** (bit en 1) |
| `jne etiqueta` | jump if not equal | la comparación dio **distinto** (bit en 0) |

⚠ **Complemento (por el título del video):** el bit en cuestión es el **zero flag** (ZF). `je` y `jz` (jump if zero) son la misma instrucción con dos nombres; lo mismo `jne` y `jnz` (jump if not zero). `cmp` hace internamente una resta sin guardar el resultado: si los valores son iguales, la resta da cero y ZF queda en 1.

### 9.2 Ejemplo corto

```
    mov  rax, 99        ; RAX = 99
    cmp  rax, 99        ; compara RAX con 99
    je   dio_igual      ; si dio igual, salta a la etiqueta
    inc  rax            ; (solo se ejecuta si NO saltó) RAX = RAX + 1
dio_igual:
    ...                 ; el programa sigue acá
```

- Como RAX vale 99, la comparación da igual, el bit se pone en 1 y **`je` salta**: el `inc rax` no se ejecuta.
- Si no hubiera dado igual, el bit quedaría en 0, no se saltaría y se ejecutaría `inc rax` (incrementa RAX en uno).
- La etiqueta `dio_igual` corresponde a una dirección de memoria; se ejecuta el código que está de ahí hacia abajo.

## 9.3 Ejemplo generado desde C

```
int main() {
    int valor = 45;
    int no_hace_nada = 0;

    if (valor != 20) {
        no_hace_nada++;
        no_hace_nada--;
    }
    if (valor == 55) {
        no_hace_nada++;
        no_hace_nada--;
    }
}
```

Sumar y restar uno no tiene ningún objetivo: es solo para que haya instrucciones dentro del `if`. (En assembly existe `nop`, una instrucción que no hace nada.)

```
main:
    push rbp
    mov  rbp, rsp
    mov  DWORD PTR [rbp-4], 45     ; valor = 45
    mov  DWORD PTR [rbp-8], 0      ; no_hace_nada = 0

    cmp  DWORD PTR [rbp-4], 20     ; compara valor con 20
    je   .L2                       ; si es IGUAL, saltea el bloque del if
    add  DWORD PTR [rbp-8], 1      ; no_hace_nada++
    sub  DWORD PTR [rbp-8], 1      ; no_hace_nada--
.L2:
    cmp  DWORD PTR [rbp-4], 55     ; compara valor con 55
    jne  .L3                       ; si es DISTINTO, saltea el bloque del if
    add  DWORD PTR [rbp-8], 1      ; no_hace_nada++
    sub  DWORD PTR [rbp-8], 1      ; no_hace_nada--
.L3:
    mov  eax, 0
    pop  rbp
    ret
```

## 9.4 El compilador pregunta por la condición opuesta

| En C | En assembly | Lógica |
|---|---|---|
| `if (valor != 20) { bloque }` | `cmp …, 20` y `je .L2` | "Si es **igual**, saltá el bloque" |
| `if (valor == 55) { bloque }` | `cmp …, 55` y `jne .L3` | "Si es **distinto**, saltá el bloque" |

- En C se pregunta por la condición para **entrar** al bloque.
- En assembly es más simple preguntar por la condición para **saltearlo**: si se cumple lo opuesto, se salta por encima de las instrucciones del bloque hasta la etiqueta; si no, se sigue de largo y se ejecutan.
- Es la forma óptima de traducir un `if`.

Seguimiento con `valor = 45`:

1. `cmp` con 20 → son distintos → `je` **no salta** → se ejecutan `add` y `sub`.
2. `cmp` con 55 → son distintos → `jne` **salta** a `.L3` → el segundo bloque no se ejecuta.
3. En `.L3`: se carga 0 en EAX, `pop rbp` y `ret` para volver al sistema operativo.

## 9.5 Otras familias de saltos

Además de `je` y `jne` hay muchas más instrucciones de salto condicional, según qué flag miren:

- si hubo **overflow**;
- si hubo **carry**;
- si dio **igual** o **distinto**;
- si dio **cero** o **no dio cero**.

Todas funcionan igual: después de que ocurrió una operación, la instrucción de salto **analiza un bit**

**del registro de estado y, según eso, salta o no salta**. Con eso se controla la lógica del programa.

## 10. Ideas clave para repasar

- **Contador**: 4 flip-flops T con T=1, cada $\bar{Q}$ al clock del siguiente. Cuenta en binario de 0 a 15 (16 valores).
- **Buses**: direcciones (solo sale del CPU), datos (ida y vuelta), control (leer/escribir y selección).
- **ALU**: operaciones aritméticas y lógicas. **Registro**: conjunto de flip-flops; "una variable por hardware" dentro del micro. **Lógica de control**: decodifica la instrucción.
- **Ciclo de una instrucción**: traer → decodificar → buscar operandos → operar en la ALU → guardar el resultado → siguiente.
- **Arquitectura PC**: basada en Intel, con la promesa de compatibilidad hacia atrás. Los 64 bits compatibles los inventó **AMD**.
- **Modos**: real (16 bits, al encender o resetear), protegido (segmentación y paginación, acceso a toda la RAM), compatibilidad y 64 bits.
- **Nombres**: AL/AH (8) → AX (16) → EAX (32) → RAX (64). Byte, word, double word, quad word.
- **Propósito general**: A acumulador, B base, C contador, D datos, SI/DI origen y destino, BP base de pila, SP puntero de pila.
- **Segmento** (siempre 16 bits): CS código, DS datos, SS pila, ES/FS/GS auxiliares.
- **FLAGS**: bits de estado (cero, signo, carry, overflow, paridad, interrupciones). **IP**: dirección de la próxima instrucción.
- **Pila** = variables creadas en el código, se pierden al salir de la función. **Heap** = memoria dinámica (`malloc`, `new`).
- Las variables son direcciones relativas a RBP: `[rbp-4]`, `[rbp-8]`…
- **Toda función**: `push rbp` / `mov rbp, rsp` al entrar; `pop rbp` / `ret` al salir. El valor de retorno va en **EAX**.
- **Aritmética**: `add`, `sub`, `imul`, `idiv`. Se traen los operandos a registros, se opera y se guarda EAX en memoria.
- **`call`** = salto con retorno a una etiqueta. **`ret`** = volver. El parámetro viaja en EDI (convención).
- **Salto condicional**: `cmp` modifica los flags; `je` / `jne` miran un bit y saltan o no. El compilador pregunta por la condición **opuesta** a la del `if`.